

Implementing Liu *et al.* (IEEE IoT-J 2022) as a Baseline for the Fleet-Sizing Rule: Required Changes, Procedure and Results

Implementation report

October 7, 2026

Abstract

We re-implemented the multi-population genetic algorithm (MPGA) of Liu, Guo, Li and Song, “AoI-Minimal Task Assignment and Trajectory Optimization in Multi-UAV-Assisted IoT Networks” (IEEE IoT-J, 2022), as an external baseline for the fleet-sizing paper. We adapted it to that paper’s system model: one shared energy inventory split over K UAVs, nonstop sortie operation from a central depot, and priority-weighted age at collection. This report lists every change needed and why, the exact steps followed, the tests, and the results on the three layout families (clustered “paper”, ring, core) at $M = 50$ sensors and $E_{\max} = 1.5$ MJ. We compare against our simulated-annealing (SA) planner, with the published cluster patrol as a reference. Across the three families, the adapted Liu planner’s optimal fleet size lies within one UAV of the closed-form point predictor ℓ in 36/36 instances, and its time-average AoI at its own optimum is lower than SA’s in 22/36 instances.

1 The two methods being compared

Our planner (the original). K UAVs share a usable energy inventory $U = (1 - \rho)E_{\max}$, so each sortie has a budget of U/K . Every time a UAV lands it immediately relaunches and plans one sortie. The planner solves a set-orienting problem: maximise $\sum_j w_j \hat{a}_j$ over sensors not already committed to another UAV (depot-mediated exclusion), subject to the sortie budget. It uses simulated annealing with 1,200 iterations, a cost-aware eject-and-insert repair move, and a greedy seed. The fleet-sizing rule gives the point predictor $\ell = \min(\lfloor K_{\text{reach}} \rfloor, \text{round } K_{\text{commute}})$.

Liu *et al.* (the baseline). UAVs leave a data center, collect from their assigned sensors in order, meet at an interaction point (IPT) to share data, deliver it to users, and return. The objective is the average AoI of the data the users receive, subject to a per-UAV energy limit and a minimum separation between UAVs. The joint assignment-and-routing subproblem is solved by an MPGA. A chromosome is one permutation of all sensors (followed by all users), cut into one segment per UAV by breakpoints. The GA runs $N_p = 8$ populations of $N_0 = 60$ individuals for $N_{\text{iter}} = 100$ generations. It uses roulette selection on a normalised fitness, with the population’s or the global best sometimes chosen as a parent; order (sequential) crossover; swap, inversion and slide mutations; and a breakpoint-update operator that balances route lengths. Each population has its own $p_{cr} \sim \mathcal{U}[0.5, 0.95]$ and $p_{mu} \sim \mathcal{U}[0.05, 0.3]$. The paper also gives an improved global K-means + dynamic-programming hybrid, and a Fermat–Weber step that places the IPT.

2 Changes required to implement Liu *et al.* in our model

2.1 Components removed

Liu component	Reason for removal
Users, IPT, UAV-to-UAV data sharing, delivery stage, IPT placement (CVX)	Our model has no users. Age resets when the UAV uploads a sensor’s buffer (age at collection).
Collision avoidance (stop-wait, altitude change)	Not modelled for any planner in our simulator.
Acceleration correction ΔT_d	Our UAVs fly at constant speed v , which is $\Delta T_d = 0$.
FDMA and the SNR-dependent rate $B \log_2(1 + \text{SNR})$	Our UAVs hover directly above the sensor at a fixed rate R_u . Upload time is $\min(\lambda \hat{A}, B)/R_u$.
K-means + DP hybrid, GADP	Held–Karp costs $O(2^n n^2)$, which is intractable at 10–50 sensors per territory. MPGA is Liu’s scalable main method.

Table 1: Parts of Liu *et al.* that have no counterpart in our system model.

2.2 Components changed

Liu	Our adaptation	Reason
Each UAV has its own battery $E_n^{\max}$	Every sortie has U/K (shared inventory)	Without this, the fleet-sizing trade-off and K^* do not exist.
One mission visiting every assigned sensor	Segment k becomes UAV k ’s territory and its repeating visit sequence. At each launch the UAV flies the next contiguous run that fits U/K	At U/K^* a whole territory rarely fits in one sortie. The cutting rule is the same one used by cluster patrol and C4.
Infeasible chromosome $\Rightarrow$ fitness 0; task reduction	Energy is enforced by the sortie-cutting step. A sensor unreachable even on its own is skipped	As a single mission, almost every chromosome breaks the energy limit, so every fitness would be 0. Task reduction would abandon sensors that are still reachable.
Fitness = users’ average AoI (Liu eq. 22)	Fitness = time-average priority-weighted age at collection of the repeating patrol, from the planner-side surrogate <code>Ctx.uav_J</code>	Optimising a one-mission objective for a nonstop deployment would hand the baseline the wrong target. The surrogate simulates our launch stagger, hover-time estimate, cutting rule and reserve check, and integrates weighted age exactly.
User-demand weights τ	Sensor priorities w_i	The same objective for every planner.
IPT pull in the K-means center update	Depot pull	There is no IPT. The depot is the shared point every UAV returns to.
Number of UAVs N	Swept K	This is the quantity under test.

Table 2: Parts of Liu *et al.* that were changed, and why.

A correction to a common reading. Liu’s “centralized information sharing” exchanges *collected data* between UAVs at the IPT so it can be delivered to users. It does *not* stop two UAVs serving the same sensor. In Liu that is prevented only by disjoint assignment, which our adaptation keeps. The depot’s commitment exclusion is still applied when a sortie is cut.

2.3 Details the paper does not specify

Item	Value used (agreed before implementation)
Initial population	One individual from Liu’s improved K-means (each cluster ordered by nearest-neighbour + 2-opt). All others are random permutations with random breakpoints.
Best-individual parents	Population best a_i^l with $p = 0.1$, global best a_{opt} with $p = 0.1$, otherwise roulette on F .
Elitism	Each population carries over its best individual.
Mutation choice	When mutation fires (p_{mu}), swap, inversion or slide with equal probability. Slide = move a random block to a random position.
Breakpoint update	With probability p_{mu} , move one random breakpoint by 1–3 genes toward balancing the closed depot-tour lengths of the two adjacent segments.
ψ in F	10^{-9} .
p_{cr}, p_{mu} draw	Once per population, fixed RNG seed 0, so each (instance, K) is deterministic.
Improved K-means Dr_i	The printed $\sum_j d(i, j) / \sum_l d(i, l) \equiv 1$ is a typo. We use $Dr_i = \sum_j d(i, j)$ (the most central target seeds the first cluster), followed by Lloyd iterations with the depot-pulled center update.
Crossover	OX1. It reproduces both children of Liu’s Fig. 2 exactly (unit test). The child keeps parent 1’s breakpoints, as in the figure.

Table 3: Values chosen where Liu *et al.* leave details open. All are exposed in CFG.

3 Implementation

The code is in `baselines/liu2022/`. The only change elsewhere is a six-line `--planner liu_mpga` branch in `experiments/run_grid.py`. The planner uses the same interface and the same information as every other planner: sensor positions, priorities, the depot, airframe and energy parameters, and the *nominal* generation rate. It does not see true rates or events.

Algorithm 1 Adapted Liu MPGA planner inside the simulator

- 1: **On the first launch:** build the surrogate context $\mathcal{C}$ (distances, w , U/K , nominal λ , window)
 - 2: seed $\leftarrow$ improved K-means $\rightarrow$ clusters ordered by NN + 2-opt $\rightarrow$ (perm, cuts)
 - 3: run MPGA ($N_p=8$, $N_0=60$, $N_{\text{iter}}=100$) with fitness $\Delta(\text{perm}, \text{cuts}) = \frac{1}{H-T_b} \sum_k \mathcal{C}.\text{uav_J}(\text{segment}_k)$
 - 4: $\text{seq}_k \leftarrow$ segment k of the best chromosome; $\text{ptr}_k \leftarrow 0$
 - 5: **On every launch of UAV k :** route $\leftarrow$ CUTSORTIE($\text{seq}_k, \text{ptr}_k, U/K, \text{excluded}$)
 - 6: $\triangleright$ next contiguous run that fits the budget; skips sensors committed elsewhere or unreachable alone
-

Segment evaluations are cached by $(k, \text{sequence})$, because a segment’s surrogate value depends only on those two. One GA run uses about 30–40 thousand segment simulations and takes 10–40 s. A full 12 h simulation with it takes 30–80 s.

4 Exact steps followed

1. **Literature selection.** Searched IEEE Xplore (2021–2026) for multi-UAV AoI planners with one-node-at-a-time collection. Liu *et al.* was chosen because it is a heuristic (no training), has a depot take-off and return structure, and is published in the target journal.
2. **Full reading of the paper.** System model (Sec. II), P1–P4, Algorithms 1–3, and Fig. 2 (crossover, recovered from the PDF figure).
3. **Mapping onto our model.** Every Liu component was classified as kept, removed or changed (Tables 1–2).
4. **Decisions agreed before coding.** Fitness = exact patrol surrogate; GA defaults as in Table 3; comparison on existing instances so SA rows pair directly; code in a separate folder.
5. **Implementation** (`liu_mpga.py`), reusing the repository’s `tour_order`, `Ctx.uav_J` and `cut_sortie`, so nothing is duplicated.

6. **Tests on very small environments** (Section 4.1).
7. **Smoke comparison** on two seeds at $M = 40$ against stored SA results.
8. $M = 40$ **run** (paper + ring, 12 seeds, 148 runs) on exactly the keys of the stored SA results.
9. $M = 50$ **run**, all three layouts (12 seeds, 404 runs) on exactly the keys of the stored SA results (`sa_rerun.csv`). Cluster patrol from `cluster_win.csv`. SA and cluster patrol were *not* re-run.
10. **Analysis**. Per instance: each planner’s $\arg \min_K J$, the point predictor ℓ (from that planner’s own measured r_c , as in `analysis/baseline_table.py`), and the J ratios. All numbers in this report are generated from the CSVs by `report/make_report.py`.

4.1 Tests on very small environments

Test	Environment	Check (all pass)
Crossover vs Liu Fig. 2	–	Both printed children reproduced exactly.
Operator validity	$M = 12, K = 3$	3,000 random steps always give a valid permutation and sorted breakpoints.
Improved K-means	$M = 40, K = 4$	Labels in range, deterministic, valid seed chromosome.
Brute-force optimum	$M = 5, K = 2, 3$ fields	All $5! \times 6 = 720$ chromosomes enumerated. In these fields the K-means seed is 3–31% above the optimum. MPGA reaches the exact optimum in all three.
Harness identity	$M = 30, K = 3,$ 4h	With cluster patrol’s sequences injected, the planner reproduces cluster patrol’s J bit for bit.
End to end	$M = 8, K = 2,$ 4h	Finite J , every sensor served, zero duplicate visits, MPGA surrogate $\leq$ seed.

Table 4: Self-tests (`python baselines/liu2022/test_liu.py`).

5 Experimental setup

Field $L = 12.6$ km, depot at the center, $v = 20$ m/s, $P_f = 300$ W, $P_h = 400$ W, $R_u = 2$ Mb/s, $B = 50$ Mb, $\lambda_i \sim 5$ kb/s $\times \mathcal{U}(0.5, 1.5)$, $w_i \sim \mathcal{U}(1, 10)$, $\rho = 0.2$, events with $g = 5$. Horizon 12h with 3h burn-in. Coordinated, launch-time planning. $E_{\max} = 1.5$ MJ, 12 seeds per layout. Layouts: *paper* (15 Gaussian hotspots holding 70% of sensors, the rest uniform), *ring* (sensors in a thin annulus), *core* (sensors near the depot plus one peripheral outlier). The K values are those of the stored SA sweeps, which were extended until every curve rises again.

Metrics. For each instance and planner: $\arg \min_K J$; agreement with ℓ (exact and within ± 1); agreement with SA’s $\arg \min$ (within ± 1); J at the planner’s own optimum divided by SA’s J at SA’s optimum (median, and how many instances the planner wins); J at SA’s predicted ℓ relative to SA’s J there; and the fraction of instances whose $\arg \min$ sits at the top of the range compared (“censored”).

6 Results

Planner	n	mean arg min	arg min exact	vs ℓ ± 1	arg min vs SA ± 1	J at own opt. vs SA	wins vs SA	J at ℓ vs SA	cens.
<i>Clustered (paper)</i>									
SA	12	4.50	92%	100%	—	—	—	—	0%
Cluster patrol	12	4.08	50%	100%	100%	+26.9%	1/12	+35.1%	0%
Liu MPGA	12	4.33	75%	100%	100%	+2.0%	4/12	+2.0%	0%
<i>Ring</i>									
SA	12	3.33	67%	100%	—	—	—	—	0%
Cluster patrol	12	2.83	33%	100%	100%	+2.9%	5/12	+16.8%	0%
Liu MPGA	12	3.00	67%	100%	92%	−0.6%	6/12	−0.9%	0%
<i>Core</i>									
SA	12	6.83	50%	83%	—	—	—	—	0%
Cluster patrol	12	5.00	67%	100%	100%	−38.2%	12/12	−39.5%	0%
Liu MPGA	12	5.00	67%	100%	83%	−40.2%	12/12	−42.1%	0%

Table 5: Primary study: $M = 50$, $E_{\max} = 1.5$ MJ, 12 seeds per layout. Negative J percentages mean lower AoI than SA. Cluster patrol was stored only up to $K = 9$ (core) and $K = 6$ (ring), so its comparison uses that shorter range. “cens.” flags arg min values at the top of the compared range.

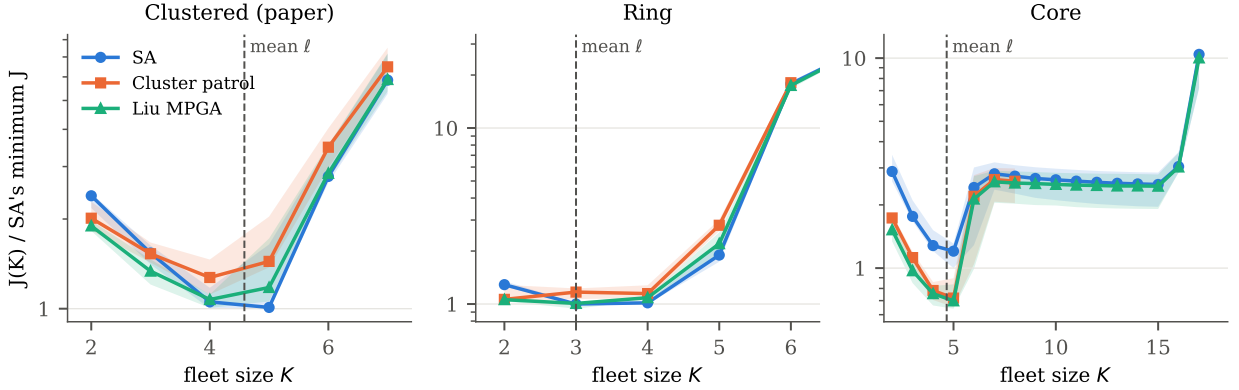


Figure 1: $J(K)$ divided by SA’s per-instance minimum (log scale). Lines are the median over 12 seeds and bands the interquartile range. Each planner is drawn on the K range it was run on. Dashed line: mean point predictor ℓ .

Layout	median $J_{\text{Liu}}/J_{\text{SA}}$ per K ($M = 50$)
Clustered (paper)	$K=2$: 0.79, $K=3$: 0.87, $K=4$: 0.98, $K=5$: 1.04, $K=6$: 1.03, $K=7$: 1.00, $K=8$: 1.01, $K=9$: 1.00
Ring	$K=2$: 0.82, $K=3$: 0.99, $K=4$: 1.04, $K=5$: 1.06, $K=6$: 0.99, $K=7$: 1.00, $K=8$: 1.00, $K=9$: 1.00, $K=10$: 1.00
Core	$K=2$: 0.52, $K=3$: 0.54, $K=4$: 0.57, $K=5$: 0.60, $K=6$: 0.87, $K=7$: 0.91, $K=8$: 0.92, $K=9$: 0.94, $K=10$: 0.95

Table 6: Paired per- K ratio of adapted Liu to SA (median over seeds), $K \leq 10$. For $K > 10$ every median lies in $[0.96, 1.00]$.

Fleet size: the predictor holds under all three planners. The adapted Liu planner’s optimal fleet size is within one UAV of the point predictor ℓ in 36/36 instances (paper 100%, ring 100%, core 100%), and exactly at ℓ in 75%, 67% and 67% of instances respectively. Cluster patrol also stays within one UAV in every instance. No planner’s optimum is censored. On the core layout, both fixed-territory planners sit *closer* to

ℓ than SA does: SA’s optimum drifts up to a mean of 6.83 UAVs against a mean of 5.00 for Liu, and SA is within one UAV of ℓ in only 83% of core instances. This upward drift is the stranding regime that the fleet-sizing paper reports for the core family (a far minimum reached by abandoning the peripheral outlier). It is a property of SA’s curve on this layout, not of the rule.

AoI level depends on the layout.

- *Clustered (paper)*: SA is better. At each planner’s own optimum, Liu’s median J is +2.0% relative to SA, and Liu wins 4/12 instances. Cluster patrol is +26.9%.
- *Ring*: essentially a tie (Liu −0.6%, wins 6/12).
- *Core*: the fixed-territory planners are far better. Liu is −40.2% and cluster patrol −38.2%, each winning 12/12 and 12/12 instances.

Per K (Table 6), Liu’s advantage is concentrated at small fleets (median ratio 0.79 at $K = 2$ on paper, 0.82 on ring, 0.52 on core). On paper and ring it disappears by $K \approx 4$. On core it persists up to $K = 5$ and then narrows as stranding sets in for every planner.

Comparison with the original paper’s own baseline findings. The fleet-sizing paper reports that the published cluster policy is about 32% worse than SA on clustered fields and about 31% better on core. Here, at $M = 50$, cluster patrol is +26.9% on paper and −38.2% on core, so the direction and size match. The adapted Liu planner keeps the core advantage of fixed territories, and on clustered fields it closes most of cluster patrol’s gap to SA (+2.0% against +26.9%). If an operator fixes the fleet at SA’s predicted ℓ , Liu’s J relative to SA is +2.0% (paper), −0.9% (ring) and −42.1% (core).

What this means for the fleet-sizing paper. The claim that the optimal fleet size is set by the rule and is stable across planners is **supported**, now including a GA-optimised territorial planner from the literature on all three layouts. A claim that SA gives the lowest AoI holds only on clustered fields at $M = 50$. It does not hold on core, where it was already known not to, or clearly on ring.

6.1 Secondary check: $M = 40$ (paper and ring)

Planner	n	mean arg min	arg min exact	vs ℓ ±1	arg min vs SA ±1	J at own opt. vs SA	wins vs SA	J at ℓ vs SA	cens.
<i>Clustered (paper)</i>									
SA	12	4.83	75%	100%	—	—	—	—	0%
Cluster patrol	12	4.33	50%	92%	92%	+6.1%	2/12	+25.6%	0%
Liu MPGA	12	4.25	50%	100%	100%	−5.5%	9/12	−2.7%	0%
<i>Ring</i>									
SA	12	3.50	50%	100%	—	—	—	—	0%
Cluster patrol	12	3.00	33%	100%	75%	+3.7%	3/12	+17.0%	0%
Liu MPGA	12	3.17	67%	100%	100%	−2.9%	10/12	−4.6%	0%

Table 7: The controlled $M = 40$ set (paper and ring), same columns as Table 5.

At $M = 40$ the fleet-size result is the same: Liu’s optimum is within one UAV of ℓ in 24/24 instances. The AoI comparison is less favourable to SA. Liu’s median J at its own optimum is −5.5% on paper (wins 9/12) and −2.9% on ring (wins 10/12). On the clustered layout, then, the sign of the SA–Liu difference changes between $M = 40$ (Liu ahead) and $M = 50$ (SA ahead). With 12 seeds per cell and differences of a few percent, the AoI ordering of SA and Liu on clustered fields should be reported as *comparable*, not as a win for either planner.

7 Caveats

- **This is an adaptation, not the authors’ code.** Every change is listed in Section 2. The two that matter most are the fitness (our surrogate) and nonstop deployment (sortie cutting).

- **Strong by construction.** The baseline is Liu’s search method optimising our objective. In structure it is a GA-searched fixed-territory patrol, of the same kind as the C4 scheduler of the fleet-sizing paper. It should not be presented as a weak baseline.
- **No live adaptation.** Territories and sequences are fixed at the first launch. Only the commitment exclusion and the hover-energy estimate react to live state.
- **Fixed GA seed.** Each (instance, K) uses RNG seed 0. Sensitivity to the seed was not measured.
- **Different M across studies.** The three-layout study uses $M = 50$, because stored SA results for core exist only at $M \in \{50, 100, 200\}$. The $M = 40$ set covers paper and ring only.

8 Reproduction

```
python baselines/liu2022/test_liu.py
python baselines/liu2022/run_m40.py baselines/liu2022/results/m40_liu.csv
python baselines/liu2022/run_m40.py baselines/liu2022/results/m50_liu.csv \
    --ref data/sa_rerun.csv --M 50 --Emax 1.5e6
python baselines/liu2022/report/make_report.py
tectonic baselines/liu2022/report/report.tex
```